Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Departamentul *Ingineria Software*

Report on laboratory work no. 5

Course: Operating Systems

Topic: The File System with Your Own Hands (Linux)

Done by: st. gr. FAF-241, Lungu Ilie

Checked by: *Relitman Patricia*

Chişinău – 2026

Contents

Purpose of the work **2**

1 PART 1 – THE TREE AND THE FILE TYPES **2**
1.1 Observe . . . 3

2 PART 2 – INODES AND LINKS **3**
2.1 An inode, and a name that is not part of it . . . 3
2.2 A hard link, then removing the "original" . . . 4
2.3 A symbolic link, then removing its target . . . 4
2.4 Directories have links too . . . 4
2.5 Size is not space . . . 4
2.6 Observe . . . 4

3 PART 3 – PERMISSIONS **5**
3.1 Files . . . 5
3.2 Directories . . . 5
3.3 Special bits and other people's files . . . 6
3.4 Observe . . . 6

4 PART 4 – DISKS AND THE KERNEL AS FILES **7**
4.1 Observe . . . 8

5 STRETCH TASKS **8**
5.1 (a) Magic numbers . . . 8
5.2 (c) Delete a file that is still open . . . 9

Conclusions **9**

PURPOSE OF THE WORK

The purpose of this laboratory work is to check, on a real Linux system, the main ideas of the lecture about file systems: the directory tree and the types of files, the inode that stays behind a name, hard and symbolic links, the permission bits, and the disks and the kernel structures that are shown to the user as files. No program is written: the work consists of commands, careful observation and short explanations. The experiments were run on **Ubuntu 24.04 LTS** (Linux kernel 6.18, a Debian-family distribution with the same layout of the file system as Debian 13) in a virtual machine, in the `bash` shell, as a normal user (the command `whoami` does not print `root`), in the folder `~/os-lab5`. The outputs are reproduced as figures, and the answers to all the *Observe* boxes and to two *Stretch* tasks are given in the corresponding parts.

```
$ whoami
ilie
$ mkdir -p ~/os-lab5 && cd ~/os-lab5
```

Figure 0.1 – The check before the start: the user is not root

The work is done as a normal user because `root` ignores most of the permission checks, and the experiments of Part 3 would give wrong results.

1 PART 1 – THE TREE AND THE FILE TYPES

```
$ ls /
bin                 dev   lib64       opt   sbin.usr-is-merged  usr
bin.usr-is-merged   etc   lost+found  proc  srv                 var
boot                home  media       root  sys
container_info.json lib   mnt         run   tmp
lib.usr-is-merged   old_root            sbin
$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx  1 root root    7 Apr 22  2024 /bin -> usr/bin
crw-rw-rw-  1 root root 1, 3 Oct 23 03:04 /dev/null
drwxr-xr-x 72 root root 4096 Oct 25 19:53 /etc
-rw-r--r--  1 root root 1150 Oct 25 19:53 /etc/passwd
drwxrwxrwt  5 root root 4096 Oct 25 19:56 /tmp
```

Figure 1.1 – The root directory and the types of some paths

```
$ ls -l /dev | head
total 0
crw-r--r-- 1 root root  10, 235 Oct 23 03:04 autofs
crw------- 1 root root  10, 234 Oct 23 03:04 btrfs-control
crw------- 1 root root   5,   1 Oct 23 03:04 console
drwxr-xr-x 3 root root        60 Oct 23 03:04 cpu
crw------- 1 root root  10, 259 Oct 23 03:04 cpu_dma_latency
lrwxrwxrwx 1 root root        13 Oct 25 19:52 fd -> /proc/self/fd
crw-rw-rw- 1 root root   1,   7 Oct 23 03:04 full
crw------- 1 root root  10, 229 Oct 23 03:04 fuse
crw------- 1 root root  10, 183 Oct 23 03:04 hwrng
$ ls -l /dev/vda
brw------- 1 root root 254, 0 Oct 23 03:04 /dev/vda
$ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
    interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=
    daa5130f2a41f7fbc8662048f3294f3d439ca7ff, for GNU/Linux 3.2.0, stripped
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin
$ du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
988K    /var/log
4.7G    /usr
```

Figure 1.2 – The beginning of /dev and the output of file

The extra names in the listing of / (`container_info.json`, `old_root`, the `*.usr-is-merged` directories) belong to this virtual machine and are not part of the standard tree. The disk `/dev/vda` was added to the listing because the first ten lines of `/dev` contain no block device.

1.1 Observe

1. **The type of each path.** The type is the first letter of the `ls -l` line (Figure 1.1):

Path	Letter	Type
`/etc`	`d`	directory
`/etc/passwd`	`-`	regular file
`/bin`	`l`	symbolic link
`/dev/null`	`c`	character device
`/tmp`	`d`	directory (the last `t` is the sticky bit, see Part 3)

2. **What is `/bin`?** A symbolic link, not a directory: `/bin -> usr/bin`. Both `ls -ld` and `file` show it. In modern Debian-family systems (Debian 13 and Ubuntu 24.04) the directories `/bin`, `/sbin` and `/lib` are links into `/usr` (the "merged `/usr`" layout), so all the programs really live under `/usr`. The target is written relative (`usr/bin`), without the leading slash, so it is searched starting from the directory that contains the link, which is `/`.

3. **A character and a block device.** `/dev/null` is a character device (`c`, major 1, minor 3); `/dev/console` (major 5, minor 1) is another one. The disk `/dev/vda` is a block device (`b`, major 254, minor 0). The difference: a character device is read and written as a stream of bytes, a block device is accessed in blocks and is the way to reach the disk. Instead of the size, `ls -l` prints the *major, minor* numbers, which identify the driver and the device.

4. **`/var/log` or `/usr`?** `/usr` is much bigger: `4.7G` against `988K`, about 5000 times (4.7 GB $\approx$ 4 900 000 KB, divided by 988 KB). `/usr` holds all the installed programs and libraries, `/var/log` only the logs of a system that has been running for a short time. The number for `/var/log` may be a little too low: `du` runs as a normal user and skips the folders it cannot read, and the error messages were hidden by `2>/dev/null`.

2 PART 2 – INODES AND LINKS

2.1 An inode, and a name that is not part of it

```
$ echo hello > notes.txt
$ stat notes.txt
  File: notes.txt
  Size: 6           Blocks: 8          IO Block: 4096   regular file
Device: 254,0   Inode: 573462      Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1001/    ilie)   Gid: ( 1002/    ilie)
Access: 2026-10-25 19:53:11.558650637 +0000
Modify: 2026-10-25 19:53:11.561212530 +0000
Change: 2026-10-25 19:53:11.561212530 +0000
 Birth: 2026-10-25 19:53:11.558650637 +0000
$ ls -li notes.txt
573462 -rw-r--r-- 1 ilie ilie 6 Oct 25 19:53 notes.txt
```

Figure 2.1 – stat and ls -li on a new file

2.2 A hard link, then removing the “original”

```
$ ln notes.txt hard.txt
$ ls -li notes.txt hard.txt
573462 -rw-r--r-- 2 ilie ilie 6 Oct 25 19:53 hard.txt
573462 -rw-r--r-- 2 ilie ilie 6 Oct 25 19:53 notes.txt
$ rm notes.txt
$ cat hard.txt ; ls -li hard.txt
hello
573462 -rw-r--r-- 1 ilie ilie 6 Oct 25 19:53 hard.txt
```

Figure 2.2 – A hard link survives rm of the first name

2.3 A symbolic link, then removing its target

```
$ ln -s hard.txt soft.txt
$ ls -li hard.txt soft.txt
573462 -rw-r--r-- 1 ilie ilie 6 Oct 25 19:53 hard.txt
573463 lrwxrwxrwx 1 ilie ilie 8 Oct 25 19:53 soft.txt -> hard.txt
$ cat soft.txt
hello
$ rm hard.txt
$ cat soft.txt ; ls -l soft.txt
cat: soft.txt: No such file or directory
lrwxrwxrwx 1 ilie ilie 8 Oct 25 19:53 soft.txt -> hard.txt
```

Figure 2.3 – A symbolic link whose target was removed

2.4 Directories have links too

```
$ mkdir d ; ls -ld d
drwxr-xr-x 2 ilie ilie 4096 Oct 25 19:53 d
$ mkdir d/x d/y ; ls -ld d
drwxr-xr-x 4 ilie ilie 4096 Oct 25 19:53 d
```

Figure 2.4 – The link count of a directory

2.5 Size is not space

```
$ echo x > one.txt ; ls -l one.txt ; du -h one.txt
-rw-r--r-- 1 ilie ilie 2 Oct 25 19:53 one.txt
4.0K    one.txt
$ truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
-rw-r--r-- 1 ilie ilie 1.0G Oct 25 19:53 big.img
0   big.img
$ rm big.img
```

Figure 2.5 – Apparent size and the space really used

The inode numbers and the device number in the figures are those of the run shown; on another disk they are different, the relations between them are the same.

2.6 Observe

1. **Inode and number of names.** In `stat` (Figure 2.1) the inode is the number after `Inode:` (here `573462`; `ls -li` prints it in the first column). The number of names of the file is `Links:` (here 1; it is the second column of `ls -l`). The name `notes.txt` is not in the inode: it is an entry in the directory, a pair (name, inode number). The inode holds the type, the owner, the permissions, the size, the times and the pointers to the data blocks.

2. **Why `hard.txt` could still be read, and what `rm` removes.** After `ln`, both names show the same inode `573462` and the count 2 (Figure 2.2): the two names are equal, there is no "original" and no "copy". `rm` calls `unlink`: it removes only *one name*, that is one directory entry, and decreases the link count ($2 \to 1$). The inode and the data are freed only when the count becomes 0 and no process has the file open. Here the count stayed 1, so `hard.txt` still gave `hello`.

3. **Why `cat soft.txt` failed, and the size of `soft.txt`.** A symbolic link is a *separate* file with its own inode (`573463`, type `l`, Figure 2.3) that contains a path as text. When it is opened, the kernel reads the text `hard.txt` and looks for that name. After `rm hard.txt` no such name exists, so the open fails with `No such file or directory`, although `soft.txt` itself still exists: it is a "dangling" link (`ls -l` still prints it with its target). The size of the link is 8 bytes because it is exactly the length of the stored path `hard.txt` ($h, a, r, d, ., t, x, t = 8$ characters, without a terminating zero). A longer target would give a bigger size. A hard link has no such problem because it points to the inode, not to a name.

4. **Where the links of a directory come from.** A new directory `d` has 2 links (Figure 2.4): (1) its own name `d` in the parent directory, and (2) the entry `.` inside `d`, which points to itself. After `mkdir d/x d/y` the count is 4: each subdirectory contains an entry `..` that points back to `d`, and this gives one more link for each of them ($2 + 2 = 4$). Ordinary files created in `d` do not change the count, because they have no `...` So the count of a directory is 2 plus the number of its subdirectories.

5. **2 bytes use 4.0K, and 1 GB uses 0.** `ls -l` prints the *size*, the number of bytes of the content: `x` and the newline are 2 bytes. `du` prints the *space on the disk*. The file system gives space only in whole blocks, and the block size is 4096 bytes ("IO Block: 4096" in `stat`), so a 2-byte file occupies one block, 4.0K. `truncate -s 1G` only sets the size in the inode and does not write any data, so no block is allocated: the file is *sparse*. It has a "hole" of 1 GB, which is read as zeros, and `du` reports 0. The space is taken only when real data are written into the hole.

3 PART 3 – PERMISSIONS

3.1 Files

```
$ echo secret > s.txt
$ chmod 600 s.txt ; ls -l s.txt
-rw------- 1 ilie ilie 7 Oct 25 19:53 s.txt
$ chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied
$ chmod u+r s.txt ; cat s.txt
secret
$ umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
0022
-rw-r--r-- 1 ilie ilie    0 Oct 25 19:53 new.txt
drwxr-xr-x 2 ilie ilie 4096 Oct 25 19:53 newdir
```

Figure 3.1 – chmod on a file and the umask

3.2 Directories

Before each command the result was predicted: with `r` only, `ls box` should work and `cat box/f` should fail; with `x` only, the opposite.

```
$ mkdir box ; echo hi > box/f
$ chmod u=r,go= box ; ls box ; cat box/f     # read only
f
cat: box/f: Permission denied
$ chmod u=x,go= box ; ls box ; cat box/f     # execute only
ls: cannot open directory 'box': Permission denied
hi
$ chmod 755 box
```

Figure 3.2 – r and x on a directory

Both predictions were correct.

3.3 Special bits and other people's files

```
$ ls -l /usr/bin/passwd ; ls -ld /tmp
-rwsr-xr-x 1 root root 64152 May 30  2024 /usr/bin/passwd
drwxrwxrwt 5 root root 4096 Oct 25 19:56 /tmp
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
$ ls -l /etc/shadow
-rw-r----- 1 root shadow 635 Oct 25 19:53 /etc/shadow
```

Figure 3.3 – setuid, sticky bit and /etc/shadow

3.4 Observe

1. **The modes as strings and one use for each.**

Mode	String	Meaning	Real use
600	`rw----`	owner reads and writes, nobody else	a private key, `~/.ssh/id_ed25519` (`ssh` refuses a key that others can read)
644	`rw-r-r-`	owner writes, everybody reads	an ordinary file, e.g. `/etc/passwd`, a web page
755	`rwxr-xr-x`	owner everything, others read and execute	a program such as `/usr/bin/ls`, or a public directory
700	`rwx---`	only the owner	a private directory such as `~/.ssh`

   Each digit is the sum $r = 4$, $w = 2$, $x = 1$, for the owner, the group and the others.

2. **Why ownership does not help with `chmod 000`.** The kernel checks the bits of *one* class: if the user is the owner, the owner bits `u` are used, and only them; the group and the others are not looked at. With mode 000 the owner bits are `--`, so `cat` gets `Permission denied`. Ownership only chooses which class applies and gives the right to change the mode, so the owner can give himself the access back with `chmod u+r`, as in Figure 3.1. Only `root` ignores the bits, which is the reason the work is done as a normal user.

3. **The umask.** The umask is `0022`. It is a mask of the bits that are *removed* from the mode requested by the program: here the write bit for the group and the others. A program asks for 666 for a new file and 777 for a directory, so $666 - 022 = 644$ gives

`-rw-r-r-` for `new.txt` and $777 - 022 = 755$ gives `drwxr-xr-x` for `newdir`, exactly as in Figure 3.1. (Strictly, the mask clears bits, it is not a subtraction, but for these values the result is the same.)

4. **The `box` experiment.** On a directory, `r` allows to *read the list of names* in it, and `x` allows to *pass through* it, that is to look up a name and reach the inode of an entry (the permission is called "search" for a directory). With `r` only, `ls box` printed the name `f`, but `cat box/f` failed, because without `x` the name cannot be resolved to the file. With `x` only, `ls box` failed (the list cannot be read), but `cat box/f` worked, because the name `f` was known and the directory could be traversed. (To use the entries normally, both `r` and `x` are needed, and that is why directories usually have 755.)

5. **The `s` and the `t`.** The `s` in `-rwsr-xr-x` is the *setuid* bit: when the program is started, the process runs with the rights of the *owner of the file* (`root`), not of the user who started it. The `t` in `drwxrwxrwt` is the *sticky* bit on a directory: everybody can create files in `/tmp`, but a file can be deleted or renamed only by its owner, by the owner of the directory or by `root`. `passwd` needs the `s` because it must change `/etc/shadow`, and this file can be read and written only by `root` (mode `rw-r---`, group `shadow`); the command `cat /etc/shadow` failed in Figure 3.3. With the setuid bit an ordinary user can change his own password through a program that is trusted to touch only his own line.

4 PART 4 – DISKS AND THE KERNEL AS FILES

```
$ lsblk
NAME  MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
zram0 253:0    0    0B  0 disk
vda   254:0    0  256G  0 disk /
vdb   254:16   0  9.6M  1 disk /opt/rclone
vdc   254:32   0  692K  1 disk /mnt/skills/public
vdd   254:48   0  5.8M  1 disk /mnt/skills/examples
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           4.0G  332K  4.0G   1% /dev/shm
/dev/vda        252G  8.6G   10G  47% /
tmpfs           2.0G     0  2.0G   0% /sys/fs/cgroup
tmpfs           1.0M     0  1.0M   0% /run/sandbox-telemetry
...
$ mount | grep " / "
/dev/vda on / type ext4 (rw,relatime,discard,no_prefetch_block_bitmaps,resv_strict,resuid
    =65534,resgid=65534)
```

Figure 4.1 – The disk, the file systems and the root mount

The output of `df -h` is shortened: the last lines (network file systems that belong to the infrastructure of the virtual machine) are replaced by dots.

```
$ grep -c processor /proc/cpuinfo
1
$ head -3 /proc/meminfo
MemTotal:        4091692 kB
MemFree:         3310464 kB
MemAvailable:    3691276 kB
$ head -5 /proc/self/status
Name:   head
Umask:  0022
State:  R (running)
Tgid:   1430
Ngid:   0
$ ls -l /proc/self/fd
total 0
lr-x------ 1 ilie ilie 64 Oct 25 19:56 0 -> pipe:[2674]
l-wx------ 1 ilie ilie 64 Oct 25 19:56 1 -> pipe:[2675]
```

```
l-wx------ 1 ilie ilie 64 Oct 25 19:56 2 -> pipe:[2675]
lr-x------ 1 ilie ilie 64 Oct 25 19:56 3 -> /proc/1431/fd
```

Figure 4.2 – The kernel shown as files in /proc

4.1 Observe

1. **The disk.** The system has one main (virtual) disk, `vda`, of 256 GB. It has *no partitions*: the whole disk is one `ext4` file system, mounted at `/`:

Disk	Size	Partitions	Mounted at	Type
`vda`	256 G	none	`/`	`ext4`

   The root file system is on `/dev/vda` and its type is `ext4` (Figure 4.1). The small read-only disks `vdb`, `vdc`, `vdd` (flag `RO` = 1) are mounted in other places and `zram0` has size 0; the `tmpfs` lines of `df` are not disks: they are file systems kept in RAM. On a laptop one would see partitions (`nvme0n1p1`, ...) instead.

2. **How full is the root file system?** `df` shows 8.6 GB used of 252 GB and 47%. The percentage looks too big, but `df` computes it as used / (used + available): 8.6/(8.6 + 10) ≈ 46–47%, and only 10 GB are available for this user in this virtual machine, so the nominal size of 252 GB does not show the real room.

3. **CPU cores and RAM.** `/proc/cpuinfo` contains one line `processor`, so the system has **1 CPU** visible. `MemTotal` is 4 091 692 kB, about **3.9 GiB** (4 GB of RAM minus the part that the kernel reserves). `MemAvailable` is the memory that programs can really get.

4. **The program name in `/proc/self/status`.** It is `head`. `/proc/self` is not a fixed folder: it always means *the process that opens it*. The file was opened by the command `head -5 /proc/self/status`, which is a process of its own (created by the shell), so the kernel shows the status of `head`, not of `bash`. In the same way `ls -l /proc/self/fd` (Figure 4.2) listed the descriptors of the `ls` process: 0 is its input, 1 and 2 are its output and errors (here pipes, because the commands were run by a script; in a terminal they point to `/dev/pts/0`), and 3 is the directory `ls` itself opened to read the list.

5 STRETCH TASKS

Two optional tasks were done: (a) magic numbers and (c) deleting a file that is still open.

5.1 (a) Magic numbers

```
$ gzip -c /etc/passwd > notes.txt ; file notes.txt ; xxd notes.txt | head -1
notes.txt: gzip compressed data, was "passwd", last modified: Sun Oct 25 19:53:11 2026,
    from Unix, original size modulo 2^32 1150
00000000: 1f8b 0808 275e de6a 0003 7061 7373 7764  ....'^.j..passwd
```

Figure 5.1 – A gzip file with the name notes.txt

The name says `.txt`, but `file` reports gzip data: it does not look at the name. It reads the first bytes of the content, the *magic number*. Here the first two bytes `1f 8b` are the signature of the gzip format. They are followed by `08` (the method deflate), `08` (the flag that the original file name is stored), four bytes of the modification time, `00`, `03` (the system: Unix) and the stored name `passwd` (the text on the right of the dump). So the bytes `1f 8b` tell `file` the real type.

5.2 (c) Delete a file that is still open

```
$ dd if=/dev/zero of=bigfile bs=1M count=500 2>/dev/null
$ df -h . | tail -1
/dev/vda        252G  9.1G  9.5G  49% /
$ exec 3< bigfile ; rm bigfile ; ls -l /proc/$$/fd/3
lr-x------ 1 ilie ilie 64 Oct 25 19:53 /proc/190/fd/3 -> /home/ilie/os-lab5/bigfile (
    deleted)
$ df -h . | tail -1
/dev/vda        252G  9.1G  9.5G  49% /
$ exec 3<&-
$ df -h . | tail -1
/dev/vda        252G  8.6G   10G  47% /
```

Figure 5.2 – rm of an open file: the space is not freed until it is closed

The link `/proc/$$/fd/3` points to the path of the file with the word `(deleted)`: the name was removed from the directory, but the shell still holds descriptor 3 on the inode, so the link count is 0 and the open count is 1. The space is not freed at `rm` (`df` still shows 9.1 G used); it is freed *only when the last descriptor is closed* with `exec 3<&-`, and then the used space falls by about 500 MB (to 8.6 G). This is the same rule as in Part 2: `rm` removes a name, and the inode is released only when nothing refers to it any more. It also explains why a program can keep writing to a log that was deleted, and the disk stays full until the program is stopped.

CONCLUSIONS

In this laboratory work the file system of Linux (Ubuntu 24.04, the same layout as Debian 13) was studied with simple commands, as a normal user. In Part 1 it was seen that the type of a file is the first letter of `ls -l` (`-`, `d`, `l`, `c`, `b`), that `/bin` is a link to `/usr/bin`, and that devices are files with the major and minor numbers instead of a size. In Part 2 it was shown that the name of a file is only an entry in a directory, and the file itself is the inode: a hard link is a second name for the same inode, `rm` only removes one name, a symbolic link is a different file that stores a path (so it can become dangling), the link count of a directory is 2 plus its subdirectories, and the size of a file is not the space it uses (a sparse file of 1 GB uses nothing).

In Part 3 the permission bits were checked for the owner, the group and the others. The experiments showed that the owner is also limited by the bits, that the umask 022 gives 644 for files and 755 for directories, that on a directory `r` lists the names and `x` allows to pass through, and that the setuid bit lets `passwd` change `/etc/shadow` while the sticky bit protects the files in `/tmp`. In Part 4 the disk was seen as one `ext4` root file system on `/dev/vda`, and the kernel as files in `/proc` that always describe the process that reads them. The stretch tasks confirmed that `file` trusts the magic bytes and not the name, and that the space of a deleted but open file is released only at the closing of the file. All these facts follow from one idea of the lecture: the file system is made of names, inodes and blocks, and the commands only show different sides of it.